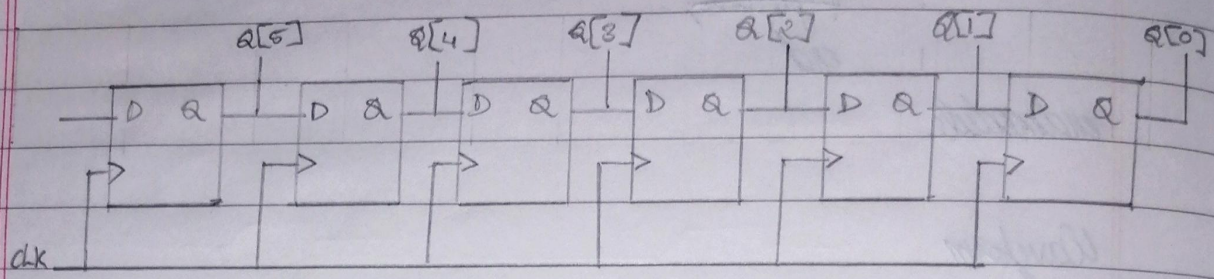
Lab No. 7
Registers

1. Write structural verilog code for 6-bit register.



Code

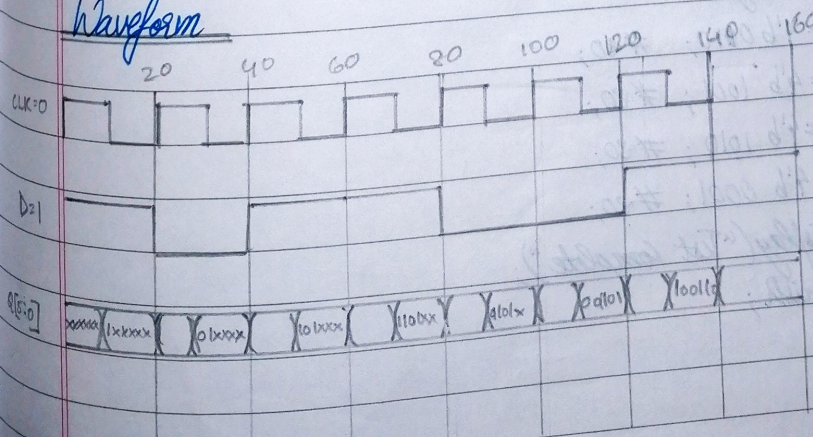
```
module DFF (D, clk, Q);
input D, clk;
output reg Q;
always @ (posedge clk)
        Q <= D;
endmodule

module Shiftreg (D, clk, Q);
input D, clk;
output [5:0] Q;
DFF  obj1 (D, clk, Q[5]);
DFF  obj2 (Q[5], clk, Q[4]);
DFF  obj3 (Q[4], clk, Q[3]);
DFF  obj4 (Q[3], clk, Q[2]);
DFF  obj5 (Q[2], clk, Q[1]);
DFF  obj6 (Q[1], clk, Q[0]);
endmodule
```

Testbench

- timescale 1ns/1ns
- include "shiftreg.v"

```
module shiftreg_tb;
    reg D, CLK;
    wire [5:0] Q;
shiftreg dup (D, CLK, Q);
initial begin
    $dumpfile("shiftreg_tb.vcd");
    $dumpvars(0, shiftreg_tb);
    CLK=0; forever #10 CLK=~CLK;
end
initial begin
    D=1; #20;
    D=0; #20;
    D=1; #20;
    D=1; #20;
    D=0; #20;
    D=0; #20;
    D=1; #20;
    $display("Test Complete"); ---> $finish;
    end
endmodule
```

Waveform



2. Write verilog code for an N-bit register.

Code

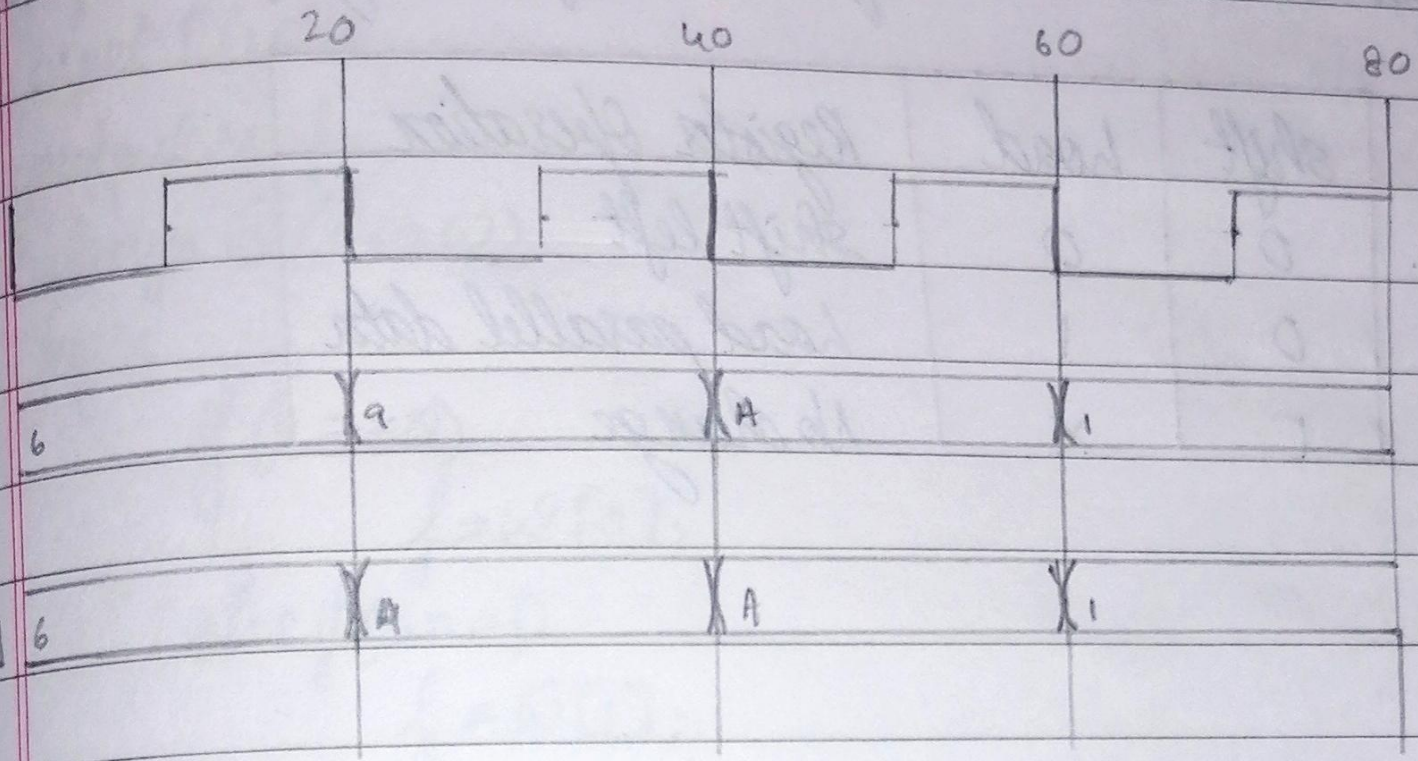
```
module nbit(D, clk, Q);
parameter n=4; integer i=0;
input [n-1:0]D;
input clk;
output reg [n-1:0] Q;
always @(negedge clk)
        Q<=D;
endmodule
```

Testbench

```
-timescale 1ns/1ns
-include "nbit.v"

module nbit_tb;
reg [3:0]D;
reg clk;
wire [3:0]Q;
nbit obj(D, clk, Q);
initial begin
        $dumpfile("nbit_tb.vcd");
        $dumpvars(0, nbit_tb);
        clk=0; forever #10 clk=~clk;
        end
        initial begin
        D=4'b0010; #20;
        D=4'b 1001; #20;
        D=4'b 1010; #20;
        D=4'b 0001; #20;
        $display("Test complete");
        $finish;
        end
endmodule
```

classmate
Date 27/10/23
Page

Waveform



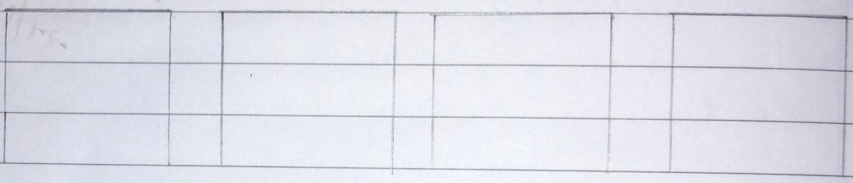
63
27/10/

Date 27/10/23
Page

3. Design and simulate a shift register with the parallel load that operates ~~accord~~ according to the following function table:

shift	load	Register Operation
0	0	shift left
0	1	Load parallel data
1	X	No change

Diagram



```
module DFF (D, clk, Reset, Q);
input D, clk, Reset;
output reg Q;
always @ (posedge clk or posedge Reset)
begin
if (Reset)
Q <= 0;
else
Q <= D;
end
endmodule
```

```
module MUX4to1(W,S,f);
input [3:0]W;
input [1:0]S;
output reg f;
always @(S or W)
begin
    if (S==0)
        f=W[0];
    else if(S==1)
        f=W[1];
    else if(S==2)
        f=W[2];
    else
        f=W[3];
    end
endmodule

`include "DFF.v"
`include "MUX4to1.v"
module Pshift(I,Reset,clk,S,A,Ser_in);
input [3:0]I;
input [1:0]S;
input Reset,clk,Ser_in;
output [3:0]A;
wire [3:0]f;
MUX4to1 mux4({A[3],A[3],I[3],A[2]},S,f[3]);
MUX4to1 mux3({A[2],A[2],I[2],A[1]},S,f[2]);
MUX4to1 mux2({A[1],A[1],I[1],A[0]},S,f[1]);
MUX4to1 mux1({A[0],A[0],I[0],Ser_in},S,f[0]);
DFF dff4(f[3],clk,Reset,A[3]);
DFF dff3(f[2],clk,Reset,A[2]);
DFF dff2(f[1],clk,Reset,A[1]);
DFF dff1(f[0],clk,Reset,A[0]);
endmodule
```

Testbench

```
`timescale 1ns/1ns
`include "Pshift.v"

module Pshift_tb;
reg [3:0] I;
reg [1:0] S;
reg Reset, clk, ser_in;
wire [3:0] A;
Pshift obj (I, Reset, clk, S, A, ser_in);
initial begin
    $dumpfile("Pshift_tb.vcd");
    $dumpvars (0, Pshift_tb);
    clk=0; forever #10 clk=~clk;
  end
  initial begin
  S= 2'b 01 ; I= 4'b 1010; Reset=0; #20;
  S= 2'b 00 ; ser_in=0; #20
  S= 2'b 00 ; ser_in=1; #20
  S= 2'b 01 ; I=4'b0001; #20;
  S=2'b 10 ; #20;
  S= $display ("Test Complete");
  end
endmodule
```

Pending